

 **Objectif**

Le but de ce TP est de découvrir comment programmer en utilisant le protocole de la couche transport UDP, via l'utilisation des sockets

**Exercice 1** *Serveur pour le service date/heure par UDP*

Un socket est une interface de programmation qui permet d'utiliser les protocoles de la couche transport.

Pour utiliser le protocole UDP par exemple, il suffit d'utiliser le constructeur `socket` avec en paramètre le type `SOCK_DGRAM`. Il est nécessaire d'associer un socket à un port, cela est possible avec la méthode `bind`

Le socket possède plusieurs méthodes pour envoyer (`sendto`) et recevoir des données (`recv`, `recvfrom`) Examinez et tapez le code suivant (pas de copier/coller). Allez sur la documentation <https://docs.python.org/3/library/socket.html> pour comprendre comment les données sont reçues et envoyées.

```
1  #!/usr/bin/python3
2
3  import socket
4  import datetime
5
6  BUFSIZE = 1024
7  def server (port ):
8      sock = socket.socket (type= socket.SOCK_DGRAM )
9      sock.bind(("0.0.0.0", port ))
10     while True:
11         data , addr = sock.recvfrom ( BUFSIZE )
12         data = datetime.datetime.now()
13         data = "%s\n" % data
14         sock. sendto (data.encode(), addr)
15
16 server(5555)
```

**Exercice 2** *Test du serveur*

Vous allez pouvoir tester ce serveur, en vous y connectant grâce à l'utilitaire en ligne de commande `netcat` :

```
netcat -4 -u localhost 5555
```

Rien ne se passe ! En effet, il faut provoquer l'envoi d'un message au serveur pour que celui-ci vous réponde. Si vous entrez une ligne de texte, netcat l'encapsulera dans un message et enverra celui-ci au serveur. Il vous suffit donc d'appuyer sur la touche Entrée. Si vous appuyez plusieurs fois, chaque fois un message sera envoyé et le serveur vous enverra une réponse.

Pour arrêter `netcat`, utilisez `Ctrl-C`.

**Exercice 3** *Client pour le service date/heure*

Le code suivant permet de développer un client qui permet de communiquer avec le serveur précédent

```
1  #!/usr/bin/python3
2  import socket
3  BUFSIZE = 1024
4  def client (host , port ):
```

```

5     sock = socket.socket(type=socket.SOCK_DGRAM )
6     addr = (host , port)
7     sock.sendto(b"", addr)
8     data = sock.recv( BUFSIZE )
9     print (data.decode (), end=" ")
10
11 client("localhost", 5555)

```

**3.1** Testez ce client en l'invokant à la ligne de commande.

#### Exercice 4 *Connexion à un serveur distant*

Le module `sys` (voir <https://docs.python.org/3/library/sys.html>) exporte la variable `argv` qui contient une liste de chaînes : ce sont les éléments de la ligne de commande. Servez- vous en pour permettre à l'utilisateur de donner sur la ligne de commande un argument indiquant le nom ou l'IP de la machine où contacter le serveur.

**4.1** Testez cette modification en interrogeant le serveur de votre voisin.

#### Exercice 5 *Serveur suite*

Pour l'instant le serveur ne sait fournir que la date. Vous allez en créer un nouveau qui saura répondre à différentes requêtes. Pour cela, vous allez dupliquer les codes que vous avez :

```

cp date-server.py ext-server.py
cp date-client.py ext-client.py

```

et vous modifierez ces nouveaux fichiers. Quelle autre information pourrait-on vouloir publier ? Par exemple l'identifiant de la personne qui fait tourner le serveur. Voici comment obtenir cette info :

```

import os
os.environ["USER"]

```

Voici comment le client envoie un message au serveur :

```
sock.sendto(b"", addr)
```

Notez que le message est vide. Pour permettre au client de faire différentes requêtes, il suffirait que le message contienne un mot indiquant l'information voulue :

```
sock.sendto(b"date", addr)
```

ou bien :

```
sock.sendto (b"user", addr)
```

De son côté le serveur doit examiner le mot reçu dans le message du client, et doit renvoyer l'information correspondante. Il faudrait que l'utilisateur puisse invoquer le client comme ceci :

```

./ext-client.py localhost date
./ext-client.py localhost user
./ext-client.py IP-DU-VOISIN date
./ext-client.py IP-DU-VOISIN user

```

où IP-DU-VOISIN serait remplacé par l'adresse IP de votre voisin, pour interroger son serveur.

#### Exercice 6 *Broadcasting server*

Il est également possible de broadcaster des messages de sorte que tous les clients d'un réseau local puissent les recevoir.

```

1  #!/usr/bin/python3
2
3  import socket
4  import datetime

```

```
5 import time
6 import os
7
8 def server(port):
9     sock = socket.socket(type=socket.SOCK_DGRAM)
10    sock.setsockopt(socket.SOL_SOCKET, socket.SO_BROADCAST, 1)
11    addr = ("255.255.255.255", port)
12    while True:
13        data = str(datetime.datetime.now())
14        data = os.environ["USER"] + " " + data
15        sock.sendto(data.encode(), addr)
16        time.sleep(1.0)
17
18
19 server(6666)
```

```
1 #! /usr/bin/python3
2
3 import socket
4
5 def client(port):
6     sock = socket.socket(type=socket.SOCK_DGRAM)
7     addr = ("255.255.255.255", port)
8     sock.bind(addr)
9     while True:
10        data, saddr = sock.recvfrom(128)
11        print(saddr, data.decode())
12
13
14 client(6666)
```